

T1.4.A — Diagnóstico Dashboard usuario

Conexión con backend · propuesta técnica + plan por PRs

Owner: Celestino · 2026-05-02 · Tipo: **read-only**, sin cambios de código

Salida: diagnóstico completo + plan por 5 PRs pequeños (T1.4.A→E). El endpoint backend (T1.4.B) reusa el patrón HMAC de T1.3.D. La sesión NextAuth ya tiene *subscription_id* que es la llave natural para identificar al usuario.

Bug P0 detectado: en `landing/auth.ts` línea 13–48, el *password* se recibe pero **nunca se valida**. Cualquier persona que conozca un email de subscriber activo puede entrar al dashboard y cancelar la suscripción. Tratar como bloqueador antes de exponer datos en T1.4.D.

1. Cómo funciona hoy el dashboard

Ruta y autenticación

- **landing/app/dashboard/page.tsx:** server component, llama `auth()`, redirige a `/login` si no hay sesión, monta `DashboardClient`.
- **landing/auth.ts:** NextAuth Credentials provider. `authorize()` lookup Stripe Customer por email + verifica subscription active. Retorna `{email, stripeCustomerId, subscriptionId, plan: priceId}`.
- **landing/app/login/page.tsx:** form email/password → `signIn('credentials')`.

UI actual (dashboard-client.tsx)

Dos secciones únicamente:

- **Suscripción:** 'Plan activo' hardcoded. Nombre del plan computa cliente comparando `session.plan` con `NEXT_PUBLIC_STRIPE_PRICE_PRO`; si no match siempre muestra 'Premium'. Texto fijo 'se renueva mensualmente' (sin fecha).
- **Conexiones:** 3 items hardcoded (Gmail, Calendar, WhatsApp) con `connected` hardcoded. Botones sin handler.

NO se muestran: créditos disponibles, créditos mensuales del plan, historial de transacciones, próxima renovación, fecha de cancelación si aplica, número de teléfono.

Cancel flow actual

`landing/app/api/cancel-subscription/route.ts` llama directo a `stripe.subscriptions.cancel(subscriptionId)`. NO notifica al backend. El backend se entera vía webhook `customer.subscription.deleted` (delay segundos a minutos).

2. Qué datos reales ya existen en backend

Tablas relevantes

Tabla	PK / Index	Datos clave
saldo_creditos	PK telefono	saldo, total_comprado, total_consumido, actualizado
transacciones_credito	PK id, idx telefono	delta, razon, stripe_session_id, saldo_resultante, creado
suscripcion_stripe	PK subscription_id	telefono, customer_id, plan_codigo, price_id, status, creditos_mensuales, ultimo_pago
evento_stripe_procesado	PK event_id	(no relevante para dashboard)

Funciones backend disponibles

- **obtener_saldo(telefono)** → **int**: saldo actual.
- **obtener_resumen(telefono)** → **dict**: saldo + total_comprado + total_consumido + últimas 5 transacciones.
Cubre prácticamente todo lo que necesita el dashboard.

3. Qué datos faltan o no están vinculados

Dato	En backend	En dashboard hoy
Saldo de créditos	saldo_creditos.saldo	No mostrado
Créditos mensuales del plan	suscripcion_stripe.creditos_mensuales	No mostrado
Plan código (premium/pro)	suscripcion_stripe.plan_codigo	Solo deduce de priceId vía env público
Status de suscripción	suscripcion_stripe.status	Hardcoded 'activa'
Total comprado / consumido	saldo_creditos.total_*	No mostrado
Últimas transacciones	transacciones_credito	No mostrado
Próxima renovación	(no en DB) — Stripe current_period_end	No mostrado
Método de pago	(no en DB) — Stripe customer.invoice_settings	No mostrado

Las fechas Stripe (próxima renovación, cancelación) se obtienen vía Stripe SDK desde el server route — más fresh que cachear en DB. Los conexión Gmail/Calendar requieren consultar UsuarioGoogleAuth en backend (out of scope para T1.4).

4. Cómo identificar al usuario

Identificador	Donde vive	Set por
email	session.user.email	Usuario lo tipea en /login
stripeCustomerId	session.stripeCustomerId	NextAuth lookup Stripe por email
subscriptionId	session.subscriptionId	NextAuth: primera sub active del customer
plan (priceId)	session.plan	NextAuth: primer item de la sub
telefono	NO está en sesión	—

El gap clave: email → telefono

El backend opera por **telefono**. La sesión solo tiene email. La única tabla que une los dos es **suscripcion_stripe**:

```
session.email
  ↓ Stripe customer.email lookup
session.stripeCustomerId (cus_xxx)
  ↓ JOIN suscripcion_stripe.customer_id = cus_xxx
suscripcion_stripe.telefono
  ↓ usar como llave en saldo_creditos / transacciones_credito
saldo, transacciones, etc.
```

Recomendación: usar subscription_id como llave

- Es la PK natural de suscripcion_stripe — lookup O(1).
- La sesión NextAuth ya lo tiene; sin round-trip extra.
- No filtra el teléfono al cliente (nunca cruza red pública si no hace falta).
- El HMAC del endpoint impide forjar IDs de otros usuarios.

5. Riesgos de seguridad

P0 (crítico) — Bug actual de auth

landing/auth.ts línea 13–48: authorize recibe email + password pero **el password nunca se valida**. Solo verifica que exista Stripe Customer con ese email + suscripción activa. Bug REAL. Quien conozca el email puede ver datos y cancelar.

Otros riesgos

Riesgo	Severidad	Mitigación
/api/cancel-subscription sin CSRF explícita	Media	Origin header check o CSRF token
Sin rate limiting en /login	Media	Rate limit por IP (brute force de emails)
subscription_id filtrable en logs	Baja	Hoy ok, no loguearlo en errores client-side
NO reusar ADMIN_TOKEN para dashboard endpoint	Info	Usar HMAC dedicado (INTERNAL_BRIDGE_SECRET)

6. Propuesta de arquitectura mínima

```
Browser
  ■ cookies (NextAuth JWT)
  ▼
Vercel · landing
  /dashboard (server component)
    auth() → session {subscriptionId, ...}
    validar password (P0 fix)
  /api/dashboard-data (T1.4.C, server-only)
    auth() → tomar subscriptionId
    POST BACKEND_URL/internal/usuario-resumen con HMAC
    merge con datos Stripe (current_period_end, etc.)
    retorna JSON al cliente
      ■ POST con HMAC
      ▼
Render · backend
  POST /internal/usuario-resumen (T1.4.B, HMAC)
  body {subscription_id: 'sub_xxx'}
  1. _verificar_firma_interna() (helper de T1.3.D)
  2. SELECT FROM suscripcion_stripe
  3. SELECT FROM saldo_creditos
  4. SELECT FROM transacciones_credito (últimas 10)
  5. JSON read-only
```

Decisiones de diseño

- **subscription_id** como llave del endpoint (no email ni teléfono).
- **HMAC reusa INTERNAL_BRIDGE_SECRET** — ya existe, ya está sincronizado en Vercel/Render. Reduce superficie de configuración.
- **Read-only**: solo SELECTs. Ningún path puede mover dinero, créditos o suscripciones desde T1.4.B.
- **Sin teléfono en la respuesta**: el dashboard no lo necesita mostrar, no viaja por red pública.
- **Stripe data desde server route**: claves Stripe nunca llegan al cliente.
- **Cancelación recomendada**: *cancel_at_period_end=true* en vez de *cancel()* immediate (preserva créditos pagados).

7. Plan por PRs pequeños

PR	Alcance	Riesgo
T1.4.A (este)	Diagnóstico (este documento). Sin código.	Cero
T1.4.B	Endpoint backend POST /internal/usuario-resumen, HMAC, read-only. Tests 8–10 casos.	Bajo (solo SELECTs)
T1.4.C	API route landing /api/dashboard-data, proxy server-side al backend con HMAC + merge de Stripe data.	Bajo
T1.4.D	UI dashboard con datos reales (saldo, plan, próxima renovación, historial). Bloqueado por CORS, requiere fix auth	Medio (CORS, auth)
T1.4.E	Cancelación: cambiar cancel() por cancel_at_period_end. Refactor pequeño.	Bajo

T1.4.B: estructura de la respuesta

```
{
  "subscription": {
    "subscription_id": "sub_xxx",
    "plan_codigo": "premium",
    "status": "active",
    "creditos_mensuales": 100,
    "ultimo_invoice_acreditado": "in_xxx",
    "creado": "2026-04-15T...",
    "actualizado": "2026-05-01T..."
  },
  "saldo": {
    "actual": 170,
    "total_comprado": 200,
    "total_consumido": 30
  },
  "ultimas_transacciones": [
    {"delta": 100, "razon": "Renovación premium...",
     "saldo_resultante": 170, "creado": "..."},
    {"delta": -2, "razon": "generar_imagen",
     "saldo_resultante": 70, "creado": "..."}
  ]
}
```

8. Tests recomendados por PR

T1.4.B (backend, 8–10 tests)

#	Test	Esperado
1	Firma inválida	401
2	Header X-Internal-Signature faltante	401
3	JSON malformado	400
4	Body sin subscription_id	400
5	subscription_id no existe	404
6	Sub existente, sin saldo	200 con saldo=0
7	Sub existente con saldo + transacciones	200 estructura completa
8	Sub canceled — devuelve igual	200 con status=canceled
9	/admin/seed-creditos legacy intacto	smoke
10	telefono NO aparece en respuesta	assert

T1.4.C / T1.4.D / T1.4.E

- **T1.4.C:** npm run build pasa, sin sesión → 401, con subscriptionId inexistente → maneja graciosamente.
- **T1.4.D:** sin framework de tests en landing — solo build + smoke manual visual.
- **T1.4.E:** tests integracionales backend al recibir customer.subscription.updated con cancel_at_period_end=true. Verificar que saldo NO se borra (regresión).
- **Auth fix P0:** password incorrecto → 401, password correcto + sub activa → ok, password correcto sin sub → mensaje específico.

9. Restricciones respetadas

- ✓ No se tocó producción.
- ✓ No se ejecutó deploy.
- ✓ No se cambiaron env vars.
- ✓ No se tocó Stripe Dashboard.
- ✓ No se expusieron secretos (solo se mencionan por nombre).
- ✓ No se implementó código.
- ✓ Reporte en Markdown + PDF en docs/auditorias/ y docs/scripts/.

10. Recomendación al owner

- **Aprobar plan T1.4.A → T1.4.E** o pedir ajustes (HMAC, estructura de respuesta, etc.).
- **Decidir prioridad del fix de password:** tratarlo como P0 antes de exponer datos en T1.4.D. Si se prefiere shipping rápido, T1.4.B + T1.4.C pueden ir antes (no afectan auth) pero T1.4.D espera fix.
- **Autorizar T1.4.B** cuando estés listo: solo SELECTs, idempotente, HMAC ya validado en T1.3.

Reporte generado el 2026-05-02 por Claude Code (Opus 4.7) en sesión con el owner. Read-only · sin cambios de código · sin deploy.